

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский
Томский политехнический университет

Инженерная школа информационных технологий и робототехники
Отделение информационных технологий

Отчет по лабораторной работе №27 по дисциплине

«Язык Kotlin и основы разработки»

Датчики

Выполнил:

Студент группы 1А22



О.К. Кравцов

Проверил:

Ст. преп. ОИТ ИШИТР



В.А. Дорофеев

Задание

Разработайте приложение, которое рисует на экране шарик, подчиняющийся закону гравитации. Для измерения гравитации используйте датчик `TYPE_GRAVITY`. При изменении положения телефона в пространстве будут изменяться и показатели гравитации, действующие вдоль каждой из осей. Используйте эти данные для расчёта скорости перемещения шарика на экране.

Попробуйте подключить датчик и организовать вывод значений в LogCat, а затем начать изменять положение телефона в пространстве, посмотрите, как будут меняться значения по каждой из трёх осей.

Когда с датчика приходят очередные данные, нужно изменить в соответствии с ними скорости по каждой из осей, а затем изменить координаты, прибавив к ним скорость.

Когда шарик долетает до одной из границ экрана, нужно чтобы он отскочил. Для этого скорость меняется на противоположную, с помощью смены знака с плюса на минус или обратно. Нужно помнить, что у шарика есть радиус, его тоже необходимо учитывать при определении долёта шарика до границы экрана.

Ход работы

1. Создан проект Lab27 на основе Empty Views Activity.
2. Настроены ресурсы приложения:

– colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="background_color">#FFFAFA</color>
    <color name="info_background">#E3F2FD</color>
    <color name="text_color">#212121</color>
    <color name="ball_color">#FF6200EE</color>
</resources>
```

– strings.xml

```
<resources>
    <string name="app_name">Lab27</string>
    <string name="gravity_sensor_data">Gravity sensor data</string>
    <string name="gravity_sensor_available">Gravity sensor: %s</string>
    <string name="gravity_sensor_not_available">Gravity sensor not available</string>
    <string name="gravity_label">Gravity</string>
    <string name="position_label">Position</string>
    <string name="velocity_label">Velocity</string>
</resources>
```

– dimens.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <dimen name="info_padding">12dp</dimen>
    <dimen name="info_text_size">14sp</dimen>
    <dimen name="info_elevation">8dp</dimen>
    <dimen name="ball_radius">20dp</dimen>
</resources>
```

3. Разработан пользовательский интерфейс:

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:app="http://schemas.android.com/apk/res-auto"
xmlns:tools="http://schemas.android.com/tools"
android:id="@+id/main"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@color/background_color"
tools:context=".MainActivity">

    <ru.olegkravtsov.lab27.BallView
        android:id="@+id/ballView"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

    <TextView
        android:id="@+id/infoText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="@color/info_background"
        android:textColor="@color/text_color"
        android:padding="@dimen/info_padding"
        android:textSize="@dimen/info_text_size"
        android:text="@string/gravity_sensor_data"
        android:elevation="@dimen/info_elevation"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

4. Реализована основная активность MainActivity:

```
package ru.olegkravtsov.lab27

import android.hardware.Sensor
import android.hardware.SensorEvent
import android.hardware.SensorEventListener
import android.hardware.SensorManager
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.widget.TextView
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity(), SensorEventListener {

    private lateinit var sensorManager: SensorManager
    private var gravitySensor: Sensor? = null
    private lateinit var ballView: BallView
    private lateinit var infoText: TextView

    private val physicsHandler = Handler(Looper.getMainLooper())
    private var isPhysicsRunning = false

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v,
insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom)
            insets
        }

        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        gravitySensor = sensorManager.getDefaultSensor(Sensor.TYPE_GRAVITY)
        ballView = findViewById(R.id.ballView)
        infoText = findViewById(R.id.infoText)

        gravitySensor?.let {
            infoText.text = getString(R.string.gravity_sensor_available, it.name)
        } ?: run {
            infoText.text = getString(R.string.gravity_sensor_not_available)
        }
    }

    override fun onResume() {
        super.onResume()
        gravitySensor?.let {
            sensorManager.registerListener(this, it,
SensorManager.SENSOR_DELAY_FASTEST)
        }
        startPhysicsLoop()
    }

    override fun onPause() {
        super.onPause()
        sensorManager.unregisterListener(this)
        stopPhysicsLoop()
    }
}
```

```

    }

    private fun startPhysicsLoop() {
        isPhysicsRunning = true
        physicsHandler.post(physicsRunnable)
    }

    private fun stopPhysicsLoop() {
        isPhysicsRunning = false
        physicsHandler.removeCallbacks(physicsRunnable)
    }

    private val physicsRunnable = object : Runnable {
        override fun run() {
            if (isPhysicsRunning) {
                ballView.updatePhysics()
                updateInfoText()
                physicsHandler.postDelayed(this, 16)
            }
        }
    }

    override fun onSensorChanged(event: SensorEvent) {
        if (event.sensor.type == Sensor.TYPE_GRAVITY) {
            ballView.setGravity(event.values[0], event.values[1])
        }
    }

    private fun updateInfoText() {
        val info = """
            ${getString(R.string.gravity_label)}: ${ballView.getGravityValues()}
            ${getString(R.string.position_label)}: ${ballView.getBallPosition()}
            ${getString(R.string.velocity_label)}: ${ballView.getVelocityValues()}
        """.trimIndent()
        infoText.text = info
    }

    override fun onAccuracyChanged(sensor: Sensor, accuracy: Int) {
    }
}

```

6. Реализация BallView:

```
package ru.olegkravtsov.lab27

import android.content.Context
import android.graphics.Canvas
import android.graphics.Paint
import android.util.AttributeSet
import android.view.View

class BallView @JvmOverloads constructor(
    context: Context,
    attrs: AttributeSet? = null,
    defStyleAttr: Int = 0
) : View(context, attrs, defStyleAttr) {

    private val ballPaint = Paint().apply {
        color = context.getColor(R.color.ball_color)
        isAntiAlias = true
    }

    private var ballX = 0f
    private var ballY = 0f
    private var ballRadius = resources.getDimension(R.dimen.ball_radius)

    private var velocityX = 0f
    private var velocityY = 0f

    private var gravityX = 0f
    private var gravityY = 0f

    private var lastPhysicsUpdate = System.currentTimeMillis()

    override fun onSizeChanged(w: Int, h: Int, oldw: Int, oldh: Int) {
        super.onSizeChanged(w, h, oldw, oldh)
        ballX = w / 2f
        ballY = h / 2f
    }

    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)
        canvas.drawCircle(ballX, ballY, ballRadius, ballPaint)
    }

    fun updatePhysics() {
        val currentTime = System.currentTimeMillis()
        val deltaTime = (currentTime - lastPhysicsUpdate) / 1000f

        if (deltaTime > 0) {
            velocityX += gravityX * deltaTime * 50
            velocityY += gravityY * deltaTime * 50

            ballX += velocityX * deltaTime * 50
            ballY += velocityY * deltaTime * 50

            checkBoundaryCollision()
            invalidate()
        }

        lastPhysicsUpdate = currentTime
    }

    private fun checkBoundaryCollision() {
        if (ballX - ballRadius < 0) {

```

```

        ballX = ballRadius
        velocityX = -velocityX * 0.8f
    } else if (ballX + ballRadius > width) {
        ballX = width - ballRadius
        velocityX = -velocityX * 0.8f
    }

    if (ballY - ballRadius < 0) {
        ballY = ballRadius
        velocityY = -velocityY * 0.8f
    } else if (ballY + ballRadius > height) {
        ballY = height - ballRadius
        velocityY = -velocityY * 0.8f
    }
}

fun setGravity(gx: Float, gy: Float) {
    gravityX = -gx
    gravityY = gy
}

fun getBallPosition(): String {
    return "X: ${ "%.2f".format(ballX) }, Y: ${ "%.2f".format(ballY) }"
}

fun getGravityValues(): String {
    return "GX: ${ "%.2f".format(gravityX) }, GY: ${ "%.2f".format(gravityY) }"
}

fun getVelocityValues(): String {
    return "VX: ${ "%.2f".format(velocityX) }, VY: ${ "%.2f".format(velocityY) }"
}
}

```


Результат работы

Разработанное приложение использует данные датчика гравитации для управления движением шара на экране. При запуске программа проверяет доступность датчика и выводит соответствующее сообщение. Шар отображается в центре экрана и начинает движение под действием гравитации при наклоне устройства. В начальном положении (телефон лежит на столе) имеется небольшое отклонение от 0 по оси Y (0,21) из-за выпирающего блока камер смартфона и небольшое отклонение по оси X (-0,12) из-за неровно стоящего стола (рис. 1).

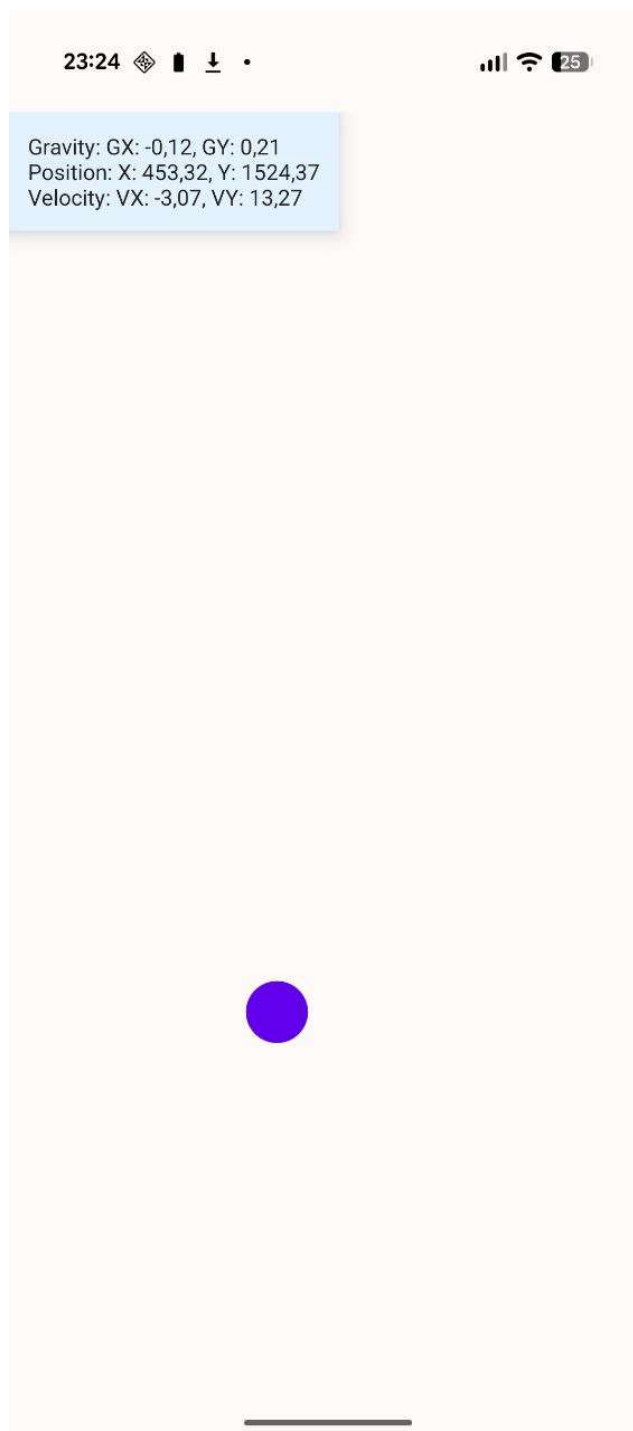


Рисунок 1 – Начальное положение

При наклоне устройства шар перемещается в соответствующем направлении. Скорость шара увеличивается под действием гравитации, а при столкновении со стенами происходит упругий удар с потерей 20% соответствующей составляющей скорости. Если наклонить устройство, то через какое-то время шар останется лежать в углу экрана (рис. 2).



Рисунок 2 – Шар в углу экрана

Выводы

В ходе лабораторной работы было разработано приложение, использующее данные датчика гравитации для реализации физической модели движения шара. В процессе работы была освоена работа с сенсорами Android через `SensorManager` и `SensorEventListener`. Реализован кастомный `View` для отрисовки и анимации графических объектов, а также создана физическая модель движения с учетом гравитации, скорости и столкновений. Для обеспечения плавной анимации был реализован непрерывный цикл обновления физики с помощью `Handler`. Организован вывод актуальной информации о состоянии системы в реальном времени, что позволяет наблюдать за изменением параметров движения шара. Приложение корректно обрабатывает изменения ориентации устройства и обеспечивает плавную анимацию движения шара, демонстрируя взаимосвязь между физическими законами и программной реализацией. Полученный опыт может быть использован для создания более сложных приложений с использованием датчиков мобильных устройств и реалистичной физической симуляции.